

Angular Moderno

30 - Controle de Acesso

2026

SENAI - CRTI

30 – Controle de Acesso

Nesta aula, vamos evoluir o projeto implementando **controle de acesso completo** na aplicação. Vamos diferenciar usuários comuns de administradores e criar uma área administrativa protegida. Também iremos melhorar a experiência do usuário quando ele não tiver permissão de acesso.

Autenticação vs Autorização

Diferenciar os dois conceitos fundamentais

Controle por Perfil

Implementar acesso baseado em perfil de usuário

Tela de Acesso Negado

Criar experiência clara de permissão negada

Área Administrativa

Evoluir a área admin com dados do usuário logado e indicadores simulados



Conceito – Autenticação vs Autorização



Autenticação

Verifica se o usuário está **logado**. É o processo de confirmar a identidade do usuário na aplicação.



Controlado pelo **authGuard**:
acesso baseado em login

Autorização

Verifica se o usuário pode **acessar determinado recurso**. É o processo de verificar as permissões após a autenticação.



Controlado pelo **adminGuard**:
acesso baseado em perfil (admin)

Na aplicação Angular, esses dois conceitos são implementados por guards distintos, cada um com uma responsabilidade clara e bem definida.

Situação Atual do Projeto

O que já existe

- Sistema de login funcional
Autenticação já implementada e operacional
- Identificação de usuário admin
Perfil de administrador já reconhecido
- Guard para rota administrativa
Proteção básica da rota `/admin` já existe

Problema Atual

⚠ Usuário comum é redirecionado para `/produtos` sem nenhuma explicação clara.

⊗ Não existe feedback visual de acesso negado para o usuário.

Nesta aula vamos **corrigir esses problemas** e melhorar significativamente a experiência do usuário.

Etapa 1 – Criar Tela de Acesso Negado

Vamos criar um novo componente para exibir quando o usuário não tiver permissão de acesso à área administrativa.

 No terminal, execute o comando abaixo para gerar o componente:

```
ng generate component features/acesso-negado/acesso-negado
```

Informar

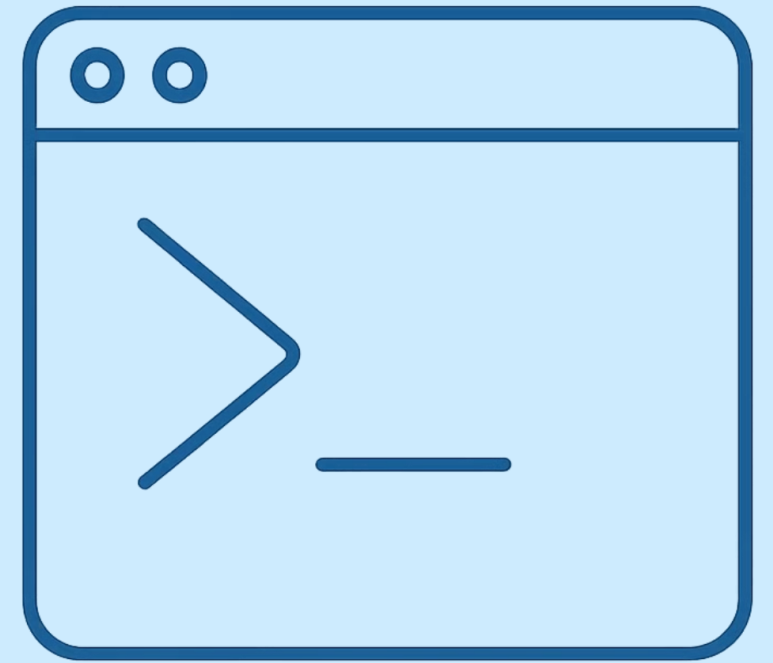
Comunicar claramente que o acesso foi negado ao usuário

Orientar

Guiar o usuário sobre o que fazer a seguir

Navegar

Permitir que o usuário volte para uma área acessível



Etapa 2: Código — `acesso-negado.ts`

`SRC/APP/FEATURES/ACESSO-NEGADO/ACESSO-NEGADO/ACESSO-NEGADO.TS`

Este componente será **standalone** e permitirá navegação com `routerLink`. Abaixo está o código completo do arquivo TypeScript:

```
import { Component } from '@angular/core';
import { RouterLink } from '@angular/router';

@Component({
  selector: 'app-acesso-negado',
  imports: [RouterLink],
  templateUrl: './acesso-negado.html',
  styleUrls: ['./acesso-negado.css'],
})
export class AcessoNegado {}
```

Standalone Component

O componente usa a nova API standalone do Angular, sem necessidade de módulos.

RouterLink importado

O `RouterLink` é importado diretamente no componente para permitir navegação declarativa no template.

Etapa 3: Código — `acesso-negado.html`

`SRC/APP/FEATURES/ACESSO-NEGADO/ACESSO-NEGADO/ACESSO-NEGADO.HTML`

Este template informa o usuário sobre a falta de permissão de acesso à área administrativa. O código abaixo representa a estrutura HTML do componente:

```
<section class="acesso-negado">
  <h2>Acesso negado</h2>

  <p>Você está autenticado, mas não possui permissão para acessar a área administrativa.</p>

  <p>Nesta aplicação, apenas usuários com perfil de administrador podem acessar essa rota.</p>

  <button type="button" routerLink="/produtos">Voltar para produtos</button>
</section>
```

📌 O botão utiliza `routerLink` para redirecionar o usuário de volta à listagem de produtos de forma segura.

Etapa 4: Código — `acesso-negado.css`

`SRC/APP/FEATURES/ACESSO-NEGADO/ACESSO-NEGADO/ACESSO-NEGADO.CSS`

Estilos simples para organizar o layout da tela de acesso negado, garantindo boa legibilidade e usabilidade:

```
.acesso-negado {  
  max-width: 600px;  
}  
  
.acesso-negado p {  
  line-height: 1.5;  
}  
  
button {  
  margin-top: 16px;  
  padding: 8px 16px;  
}
```

`max-width: 600px`

Limita a largura para melhor leitura

`line-height: 1.5`

Melhora a legibilidade do texto

`margin-top: 16px`

Espaçamento adequado no botão

Etapa 5 — Adicionar Nova Rota

SRC/APP/APP.ROUTES.TS

É necessário registrar a nova rota `/acesso-negado` no arquivo de rotas da aplicação para que o guard possa redirecionar corretamente:

```
{
  path: 'acesso-negado',
  loadComponent: () =>
    import('./features/acesso-negado/acesso-negado/acesso-negado')
    .then(m => m.AcessoNegado),
},
```

Lazy Loading

O componente é carregado de forma **lazy** com `loadComponent`, otimizando o bundle inicial da aplicação.

Rota pública

A rota `/acesso-negado` não precisa de guard, pois ela apenas informa a falta de permissão e não expõe dados sensíveis.

Integração com adminGuard

O `adminGuard` agora aponta para esta rota ao detectar usuário sem perfil de administrador.

Código Completo Para Conferência — `app.routes.ts`

`SRC/APP/APP.ROUTES.TS`

```
import { Routes } from '@angular/router';
import { authGuard } from '../core/auth.guard';
import { adminGuard } from '../core/admin.guard';

export const routes: Routes = [
  {
    path: '',
    loadComponent: () => import('../features/home/home/home').then((m) => m.Home),
  },
  {
    path: 'produtos',
    loadComponent: () => import('../features/produtos/lista-produtos/lista-produtos').then((m) => m.ListaProdutos),
  },
  {
    path: 'carrinho',
    canActivate: [authGuard],
    loadComponent: () => import('../features/carrinho/carrinho/carrinho').then((m) => m.Carrinho),
  },
  {
    path: 'checkout',
    canActivate: [authGuard],
    loadComponent: () => import('../features/checkout/checkout/checkout').then((m) => m.Checkout),
  },
  {
    path: 'admin',
    canActivate: [adminGuard],
    loadComponent: () => import('../features/admin/admin/admin').then((m) => m.Admin),
  },
  {
    path: 'acesso-negado',
    loadComponent: () => import('../features/acesso-negado/acesso-negado/acesso-negado').then((m) => m.AcessoNegado),
  },
  {
    path: 'login',
    loadComponent: () => import('../features/login/login/login').then((m) => m.Login),
  },
  {
    path: '**',
    redirectTo: '',
  },
];
```



Evoluir Área Administrativa

`SRC/APP/FEATURES/ADMIN/ADMIN/ADMIN.TS`

Objetivo: **Transformar a tela admin** em uma área mais realista e funcional, com dados do usuário logado e indicadores simulados.



Dados do Usuário

Exibir informações do usuário logado, como e-mail e perfil, diretamente na área administrativa.



Indicadores Simulados

Mostrar métricas relevantes como total de produtos, pedidos pendentes e usuários cadastrados.



Ações Administrativas

Disponibilizar ações rápidas para o administrador gerenciar a aplicação com eficiência.

Etapa 6: Código Completo — `admin.ts`

SRC/APP/FEATURES/ADMIN/ADMIN/ADMIN.TS

```
import { Component, computed, inject, signal } from '@angular/core';
import { Router } from '@angular/router';

import { AuthService } from '../../../core/services/auth.service';

@Component({
  selector: 'app-admin',
  imports: [],
  templateUrl: './admin.html',
  styleUrls: ['./admin.css'],
})
export class Admin {

  // A área administrativa passa a ler dados do usuário autenticado
  private authService = inject(AuthService);

  // Router usado para voltar para a área pública após logout
  private router = inject(Router);

  // Signals simulando indicadores administrativos do sistema.
  totalProdutosCadastrados = signal(20);
  pedidosPendentes = signal(3);
  usuariosCadastrados = signal(8);
```

```
// Computed signal derivado do usuário atual.
usuarioAtual = this.authService.usuarioAtual;

// Texto calculado para reforçar o perfil atualmente autenticado
mensagemPerfil = computed(() => {
  const usuario = this.usuarioAtual();

  if (!usuario) {
    return 'Nenhum usuário autenticado.';
  }

  return `Usuário autenticado como ${usuario.perfil}.`;
});

sair() {
  // Logout também pode ser feito a partir da área administrativa
  this.authService.logout();
  this.router.navigateByUrl('/login');
}
```

Template `admin.html`

`SRC/APP/FEATURES/ADMIN/ADMIN/ADMIN.HTML`

O template da área administrativa será enriquecido com seções de dados do usuário, indicadores e ações. Principais alterações:

Dados do Usuário

Exibir e-mail e perfil do administrador logado no cabeçalho da área admin

Indicadores

Cards com métricas: **total de produtos**, **pedidos pendentes** e **usuários cadastrados**

Ações Administrativas

Botões de ação rápida para as principais tarefas do administrador

 Os dados dos indicadores são simulados nesta etapa. Em aulas futuras, serão conectados a um backend real.

Etapa 7: Código Completo — admin.html

SRC/APP/FEATURES/ADMIN/ADMIN/ADMIN.HTML

```
<section class="painel-admin">
  <h2>Área Administrativa</h2>
  <p>Esta página representa uma área restrita da aplicação.</p>
  <p>Somente usuários com perfil de administrador conseguem acessar esta rota.</p>

  <hr />
  <section class="bloco-admin">
    <h3>Usuário autenticado</h3>

    @if (usuarioAtual()) {
      <p><strong>E-mail:</strong> {{ usuarioAtual()?.email }}</p>
      <p><strong>Perfil:</strong> {{ usuarioAtual()?.perfil }}</p>
      <p>{{ mensagemPerfil() }}</p>
    }
  </section>

  <hr />
  <section class="cards-admin">
    <article>
      <h3>Produtos cadastrados</h3>
      <p>{{ totalProdutosCadastrados() }}</p>
    </article>
    <article>
      <h3>Pedidos pendentes</h3>
      <p>{{ pedidosPendentes() }}</p>
    </article>
  </section>
</section>
```

```
<article>
  <h3>Usuários cadastrados</h3>
  <p>{{ usuariosCadastrados() }}</p>
</article>
</section>

<hr />

<section class="bloco-admin">
  <h3>Ações administrativas simuladas</h3>

  <p>
    Nesta etapa do curso, estas ações ainda são apenas representativas.
    Nas próximas evoluções, poderiam ser conectadas a APIs reais.
  </p>

  <button type="button">Cadastrar produto</button>
  <button type="button">Gerenciar pedidos</button>
  <button type="button">Gerenciar usuários</button>
</section>

<hr />

<button type="button" (click)="sair()">
  Sair da conta administrativa
</button>
</section>
```

Estilo — `admin.css`

`SRC/APP/FEATURES/ADMIN/ADMIN/ADMIN.CSS`

Objetivo: **Melhorar o layout visual** da área administrativa com um grid de cards organizado e com bom espaçamento.

Grid de Cards

Utilizar CSS Grid para organizar os indicadores em um layout responsivo e visualmente agradável.

Organização Visual

Separar claramente as seções da área administrativa: cabeçalho, indicadores e ações.

Espaçamento

Aplicar `gap` e `padding` adequados para garantir uma interface limpa e profissional.

Etapa 8: Código Completo — `admin.css`

`SRC/APP/FEATURES/ADMIN/ADMIN/ADMIN.CSS`

```
.painel-admin {
  max-width: 900px;
}

.bloco-admin {
  margin: 16px 0;
}

.cards-admin {
  display: grid;
  grid-template-columns: 1fr;
  gap: 16px;
  margin: 16px 0;
}

.cards-admin article {
  border: 1px solid #ddd;
  border-radius: 8px;
  padding: 16px;
}
```

```
.cards-admin h3 {
  margin-top: 0;
}

.cards-admin p {
  font-size: 24px;
  font-weight: bold;
}

button {
  margin-right: 8px;
  margin-top: 8px;
  padding: 8px 16px;
}

@media (min-width: 700px) {
  .cards-admin {
    grid-template-columns: repeat(3, 1fr);
  }
}
```

Melhorar Tela de Login

[SRC/APP/FEATURES/LOGIN/LOGIN/LOGIN.HTML](#)

Para facilitar os testes durante a aula, vamos adicionar uma **seção informativa** na tela de login com as credenciais de acesso do administrador.

 Adicionar seção informativa com as credenciais de teste:

Objetivo didático

Esta seção é apenas para facilitar os testes durante o desenvolvimento e as aulas. Em produção, nunca exiba credenciais na interface.

Credenciais de admin

E-mail: `admin@email.com`

Senha: qualquer senha com pelo menos 3 caracteres

Etapa 9: Código Completo — login.html

SRC/APP/FEATURES/LOGIN/LOGIN/LOGIN.HTML

```
<h2>Login</h2>

<p>Informe seus dados para simular a autenticação na loja.</p>

<section class="dica-login">
  <h3>Usuários para teste</h3>

  <p><strong>Administrador:</strong> admin@email.com</p>

  <p><strong>Usuário comum:</strong> qualquer outro e-mail válido.</p>

  <p>A senha precisa ter pelo menos 3 caracteres.</p>
</section>

@if (erroLogin()) {
  <p style="color: red">Não foi possível realizar o login.</p>
}

<form [formGroup]="formulario">
  <p><input type="email" formControlName="email" placeholder="E-mail" /></p>

  @if (formulario.get('email')?.touched && formulario.get('email')?.invalid) {
    @if (formulario.get('email')?.errors?.['required']) {
      <small>E-mail é obrigatório</small>
    }

    @if (formulario.get('email')?.errors?.['email']) {
      <small>E-mail inválido</small>
    }
  }
}
```

```
<p><input type="password" formControlName="senha" placeholder="Senha" /></p>

@if (formulario.get('senha')?.touched && formulario.get('senha')?.invalid) {
  @if (formulario.get('senha')?.errors?.['required']) {
    <small>Senha é obrigatória</small>
  }

  @if (formulario.get('senha')?.errors?.['minlength']) {
    <small>Senha deve ter no mínimo 3 caracteres</small>
  }
}

<p><button type="button" (click)="entrar()" [disabled]="formulario.invalid">
  Entrar</button>
</p>
</form>
```



Etapa 10 – Testes do Sistema

Após implementar todas as etapas, é fundamental validar o comportamento do sistema com os três cenários de teste abaixo:

1

Usuário Comum

Faça login com um e-mail comum e tente acessar `/admin`.



Resultado esperado: redirecionamento para `/acesso-negado`

2

Administrador

Faça login com `admin@email.com` e acesse `/admin`.



Resultado esperado: acesso permitido à área administrativa

3

Não Autenticado

Sem estar logado, tente acessar diretamente `/admin`.



Resultado esperado: redirecionamento para a tela de login

Conclusão da Aula

Nesta aula implementamos um sistema completo de controle de acesso em Angular Moderno.



Autenticação vs Autorização

Diferença entre verificar identidade e verificar permissões



Guards de Acesso

Uso de `authGuard` e `adminGuard` para controle de rotas



Acesso Negado

Criação de experiência clara e orientativa para permissão negada



Área Administrativa

Estruturação de painel admin com dados e indicadores



Dúvidas

?

